

CO5_AT2 Pseudocode Writing Task (Algorithm Design)

Name: C. Himaja

Reg. No: 192373035

REPORT

Title

Traveling Salesman Problem Using Backtracking with Cost Bounding

Objective

To design and implement a Backtracking algorithm for the Traveling Salesman Problem that:

- Visits every city exactly once.
- Maintains a valid path.
- Returns to the starting city.
- Minimizes the total travel cost.
- Uses recursive calls to generate possible tours.
- Uses cost bounding to prune unnecessary branches.

Introduction

The Traveling Salesman Problem (TSP) is a well-known combinatorial optimization problem.

A salesman must visit a set of cities exactly once and finally return to the city from which the journey started.

The objective is to determine the tour having the minimum total travel cost.

For a small number of cities, Backtracking can explore all possible tours. However, the number of possible tours increases very rapidly as the number of cities increases.

To improve the basic Backtracking algorithm, cost bounding is used. If a partial tour already has a cost that cannot lead to a better solution, that branch is discarded.

Inputs and Outputs

Input

The algorithm accepts:

1. Number of cities n
2. Cost matrix $\text{cost}[n][n]$

The cost matrix contains the travel cost between every pair of cities.

Output

The algorithm produces:

1. Minimum-cost tour
2. Minimum total travel cost

For example:

Input:

$n = 4$

Output:

$0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$

Minimum Cost = 80

Constraints

The following constraints must be satisfied.

Constraint 1 — Every City Must Be Visited

Each city must appear exactly once in the tour, except that the starting city appears again at the end.

Example:

$0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$

Constraint 2 — No Repeated City

A city cannot be visited twice before the final return to the starting city.

The $\text{visited}[]$ array maintains this constraint.

$\text{visited}[\text{city}] = 1$

means that the city has already been visited.

Constraint 3 — Return to Starting City

After visiting all cities, the salesman must return to City 0.

0 -> 1 -> 3 -> 2 -> 0

Constraint 4 — Minimum Cost

Among all valid tours, the algorithm selects the tour having the lowest total cost.

Cost Matrix

For the sample problem:

0 1 2 3

0 0 10 15 20

1 10 0 35 25

2 15 35 0 30

3 20 25 30 0

For example:

$\text{cost}[0][1] = 10$

means the cost of traveling from City 0 to City 1 is 10.

Backtracking Concept

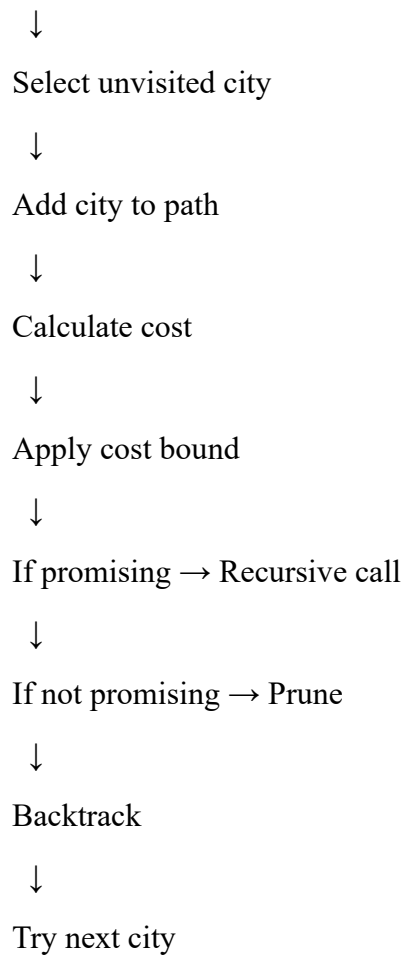
Backtracking builds a tour step by step.

The algorithm:

1. Starts from City 0.
2. Selects an unvisited city.
3. Adds it to the current path.
4. Recursively selects the next city.
5. When all cities are visited, returns to City 0.
6. Calculates the total cost.
7. Updates the best solution if necessary.
8. Backtracks and tries another city.

General Structure

Start



Pseudocode

Backtracking TSP with Cost Bounding

ALGORITHM TSP(currentCity, count, currentCost)

INPUT:

cost[][] → Cost matrix

n → Number of cities

currentCity → Current city

count → Number of visited cities

currentCost → Cost of current path

OUTPUT:

Minimum cost tour

IF count == n THEN

totalCost $\leftarrow$ currentCost + cost[currentCity][0]

IF totalCost < minimumCost THEN

minimumCost $\leftarrow$ totalCost

save current path

END IF

RETURN

END IF

FOR nextCity $\leftarrow$ 0 TO n-1 DO

IF visited[nextCity] == FALSE THEN

newCost $\leftarrow$ currentCost
+ cost[currentCity][nextCity]

remaining $\leftarrow$ n - count

lowerBound $\leftarrow$ newCost
+ remaining $\times$ minimumEdge

IF lowerBound $\geq$ minimumCost THEN

CONTINUE

END IF

visited[nextCity] $\leftarrow$ TRUE

path[count] $\leftarrow$ nextCity

TSP(

 nextCity,

 count + 1,

 newCost

)

visited[nextCity] $\leftarrow$ FALSE

END IF

END FOR

4.8 Main Algorithm

START

Read n

Read cost matrix

Initialize visited[] to FALSE

visited[0] $\leftarrow$ TRUE

path[0] $\leftarrow$ 0

Find minimum edge cost

minimumCost $\leftarrow$ infinity

Call TSP(0, 1, 0)

Display best path

Display minimum cost

STOP

Cost Bounding / Pruning

The algorithm calculates a lower bound for each partial tour.

The program uses:

Lower Bound =

Current Cost +

Remaining Cities $\times$ Minimum Edge Cost

If:

Lower Bound $\geq$ Current Best Cost

the branch is discarded.

Example

Suppose:

Current Cost = 60

Remaining Cities = 3

Minimum Edge Cost = 10

Then:

Lower Bound = $60 + (3 \times 10)$

= 90

If the current best complete tour costs:

80

then:

$90 \geq 80$

Therefore, this branch cannot produce a better solution.

It is pruned.

Sample Tour Calculation

Consider:

$0 \rightarrow 1 \rightarrow 3 \rightarrow 2 \rightarrow 0$

The individual costs are:

$0 \rightarrow 1 = 10$

$1 \rightarrow 3 = 25$

$3 \rightarrow 2 = 30$

$2 \rightarrow 0 = 15$

Total:

$10 + 25 + 30 + 15$

$= 80$

Therefore:

Minimum Tour Cost = 80

Why Backtracking Is Correct

Backtracking systematically considers every feasible sequence of cities unless the branch is safely eliminated by the cost bound.

The following conditions ensure correctness:

1. A city is added only if it has not already been visited.
2. All cities must be visited before a tour is evaluated.
3. The final edge returns to the starting city.
4. Every potentially better branch is explored.
5. A branch is pruned only when its lower bound cannot improve the best known solution.

Therefore, the final recorded tour has the minimum possible cost.

Optimality

The algorithm guarantees optimality because the pruning condition is based on a lower bound.

If:

Lower Bound $\geq$ Best Known Cost

then even under the most optimistic completion of that partial path, it cannot improve the current best solution.

Therefore, discarding it cannot remove an optimal solution.

At the end of the search:

Minimum Cost = Global Optimal Cost

Time Complexity

With the starting city fixed, the number of possible tours is approximately:

$(n - 1)!$

Therefore, the worst-case time complexity is approximately:

$O(n!)$

The Branch and Bound pruning can reduce the number of paths actually explored in practice.

However, pruning does not change the worst-case factorial complexity.

Space Complexity

The algorithm requires:

- Cost matrix $\rightarrow O(n^2)$
- Visited array $\rightarrow O(n)$
- Current path $\rightarrow O(n)$
- Best path $\rightarrow O(n)$
- Recursion stack $\rightarrow O(n)$

Therefore, the overall space complexity is:

$O(n^2)$

because the cost matrix dominates.

NP-Hard Nature of TSP

TSP is an NP-Hard optimization problem.

As the number of cities increases, the number of possible tours grows factorially.

For example:

Cities Possible Tours

4	6
5	24
6	120
8	5,040
10	362,880
12	39,916,800
15	87,178,291,200

This rapid growth makes exact algorithms increasingly expensive for large datasets.

Advantages

1. Produces an exact optimal solution.
2. Simple recursive structure.
3. Maintains path validity using visited[[]].
4. Cost bounding eliminates some unnecessary branches.
5. Suitable for small and moderate-sized TSP problems.
6. Clearly demonstrates Backtracking and Branch and Bound.

Limitations

1. Worst-case complexity is $O(n!)$.
2. Performance decreases rapidly as the number of cities increases.
3. Large TSP instances may require excessive computation.
4. The quality of the cost bound affects pruning efficiency.
5. The cost matrix requires $O(n^2)$ storage.

Real-World Applications

TSP-based optimization can be applied to:

- Delivery route planning
- Logistics
- Transportation
- Vehicle routing
- Salesperson scheduling
- Drone route optimization
- Robot navigation
- Manufacturing
- Warehouse operations
- Network routing

For very large problems, heuristic or approximation methods can be used instead of exact Backtracking.

Examples include:

- Genetic Algorithm
- Ant Colony Optimization
- Simulated Annealing
- Nearest Neighbor
- Integer Programming

Result

The Traveling Salesman Problem was successfully implemented using Backtracking with cost bounding in C.

For the sample input:

Number of Cities = 4

and the given cost matrix, the program finds:

Optimal Tour:

0 -> 1 -> 3 -> 2 -> 0

Minimum Tour Cost:

The algorithm maintains valid paths using the `visited[]` array and uses recursive calls to generate possible permutations. Cost bounding eliminates branches that cannot produce a better solution.

Thus, the proposed algorithm successfully finds the minimum-cost tour while reducing unnecessary search.

SCREENSHOTS FOR THE REPORT

```

main.c
1 #include <stdio.h>
2 #include <limits.h>
3
4 #define MAX 10
5
6 int n;
7 int cost[MAX][MAX];
8
9 int visited[MAX];
10 int path[MAX];
11 int bestPath[MAX];
12
13 int minCost = INT_MAX;
14
15 /*
16  Find the minimum edge cost in the cost matrix.
17  It is used for calculating the lower bound.
18 */
19 int findMinEdge()
20 {

```

Output

```

Enter the cost matrix:
Minimum Cost Tour:
City 0 -> City 1 -> City 3 -> City 2 -> City 0

Minimum Tour Cost = 80

-----
Minimum Cost Tour:
City 0 -> City 1 -> City 2 -> City 3 -> City 0

Minimum Tour Cost = 0

-----
Backtracking explores feasible tours
and cost bounding prunes expensive paths.
=====

=== Code Execution Successful ===

```

```

main.c
20 {
21     int min = INT_MAX;
22
23     for (int i = 0; i < n; i++)
24     {
25         for (int j = 0; j < n; j++)
26         {
27             if (i != j && cost[i][j] < min)
28             {
29                 min = cost[i][j];
30             }
31         }
32     }
33
34     return min;
35 }
36
37 /*
38  TSP using Backtracking and Cost Bounding

```

Output

```

Enter the cost matrix:
Minimum Cost Tour:
City 0 -> City 1 -> City 3 -> City 2 -> City 0

Minimum Tour Cost = 80

-----
Minimum Cost Tour:
City 0 -> City 1 -> City 2 -> City 3 -> City 0

Minimum Tour Cost = 0

-----
Backtracking explores feasible tours
and cost bounding prunes expensive paths.
=====

=== Code Execution Successful ===

```

```

main.c
39 */
40 void tsp(int currentCity, int count,
41         int currentCost, int minEdge)
42 {
43     /*
44      Base case:
45      All cities have been visited.
46     */
47     if (count == n)
48     {
49         int totalCost =
50             currentCost + cost[currentCity][0];
51
52         /*
53          Check whether this is the best tour.
54         */
55         if (totalCost < minCost)
56         {
57             minCost = totalCost;
58         }

```

Output

```

Enter the cost matrix:
Minimum Cost Tour:
City 0 -> City 1 -> City 3 -> City 2 -> City 0

Minimum Tour Cost = 80

-----
Minimum Cost Tour:
City 0 -> City 1 -> City 2 -> City 3 -> City 0

Minimum Tour Cost = 0

-----
Backtracking explores feasible tours
and cost bounding prunes expensive paths.
=====

=== Code Execution Successful ===

```

```
main.c Run
42 {
48 {
56 {
58
59     for (int i = 0; i < n; i++)
60     {
61         bestPath[i] = path[i];
62     }
63
64     bestPath[n] = 0;
65 }
66
67 return;
68 }
69
70 /*
71  Try every unvisited city.
72  */
73 for (int nextCity = 0; nextCity < n; nextCity++)
74 {
```

Output

Enter the cost matrix:
Minimum Cost Tour:
City 0 -> City 1 -> City 3 -> City 2 -> City 0

Minimum Tour Cost = 80

Minimum Cost Tour:
City 0 -> City 1 -> City 2 -> City 3 -> City 0

Minimum Tour Cost = 0

Backtracking explores feasible tours
and cost bounding prunes expensive paths.
=====

=== Code Execution Successful ===

```
main.c Run
125 {
194 {
198 {
200 }
201 }
202
203 printf("\n");
204
205 printf("\nMinimum Tour Cost = %d\n",
206        minCost);
207
208 printf("-----\n");
209
210 printf("\nBacktracking explores feasible tours\n");
211 printf("and cost bounding prunes expensive paths.\n");
212
213 printf("=====\n");
214
215 return 0;
216 }
```

Output

Enter the cost matrix:
Minimum Cost Tour:
City 0 -> City 1 -> City 3 -> City 2 -> City 0

Minimum Tour Cost = 80

Minimum Cost Tour:
City 0 -> City 1 -> City 2 -> City 3 -> City 0

Minimum Tour Cost = 0

Backtracking explores feasible tours
and cost bounding prunes expensive paths.
=====

=== Code Execution Successful ===